

Backend-Testing & TDD

Modul: M324 – DevOps-Prozesse mit Tools unterstützen

Autor: Metehan Celik

Unit-Tests für bestehenden Code

Als erstes wurden Unit-Tests für den bereits bestehenden Code geschrieben. Ziel war es, das bestehende Verhalten des Backends zu prüfen.

Test 1 – Anwendung startet korrekt

Prüft ob der Spring-Kontext fehlerfrei geladen wird.

```
@Test
void contextLoads() {
    assertTrue(true);
}
```

Test 2 – Task Getter/Setter

Prüft ob die `Task`-Klasse den Wert korrekt speichert und zurückgibt.

```
@Test
void testTaskDescription() {
    Task task = new Task();
    task.setTaskdescription("Einkaufen");
    assertEquals("Einkaufen", task.getTaskdescription());
}
```

Test 3 – GET-Endpoint mit MockMvc

Prüft ob der REST-Endpoint `GET /` mit HTTP 200 antwortet.

```
@Test
void testGetEndpointReturnsOk() throws Exception {
    mockMvc.perform(get("/"))
        .andExpect(status().isOk());
}
```

```
13 @SpringBootTest
14 @AutoConfigureMockMvc
15 class DemoApplicationTests {
16
17     @Autowired
18     private MockMvc mockMvc;
19
20     // Test 1: Anwendung startet korrekt
21     @Test
22     void contextLoads() {
23         assertTrue(true);
24     }
25
26     // Test 2: Task Getter/Setter funktioniert
27     @Test
28     void testTaskDescription() {
29         Task task = new Task();
30         task.setTaskdescription(taskdescription: "Einkaufen");
31         assertEquals("Einkaufen", task.getTaskdescription());
32     }
33
34     // Test 3: GET "/" gibt HTTP 200 zurück (MockMvc)
35     @Test
36     void testGetEndpointReturnsOk() throws Exception {
37         mockMvc.perform(get(uriTemplate: "/"))
38             .andExpect(status().isOk());
39     }
40
41     // TDD Test 4: Task hat Erfassungsdatum bei Instanziierung
42     @Test
43     void testTaskHasCreationDate() {
44         Task task = new Task();
45         assertNotNull(task.getCreatedAt(), "Task soll ein Erfassungsdatum haben");
46     }
47
48 }
```

TDD – Test Driven Development

Schritt 1: Failing Test schreiben

Zuerst wurde ein Test geschrieben für eine Funktionalität, die noch nicht existiert: das automatische Speichern des Erfassungsdatums beim Erstellen eines Tasks.

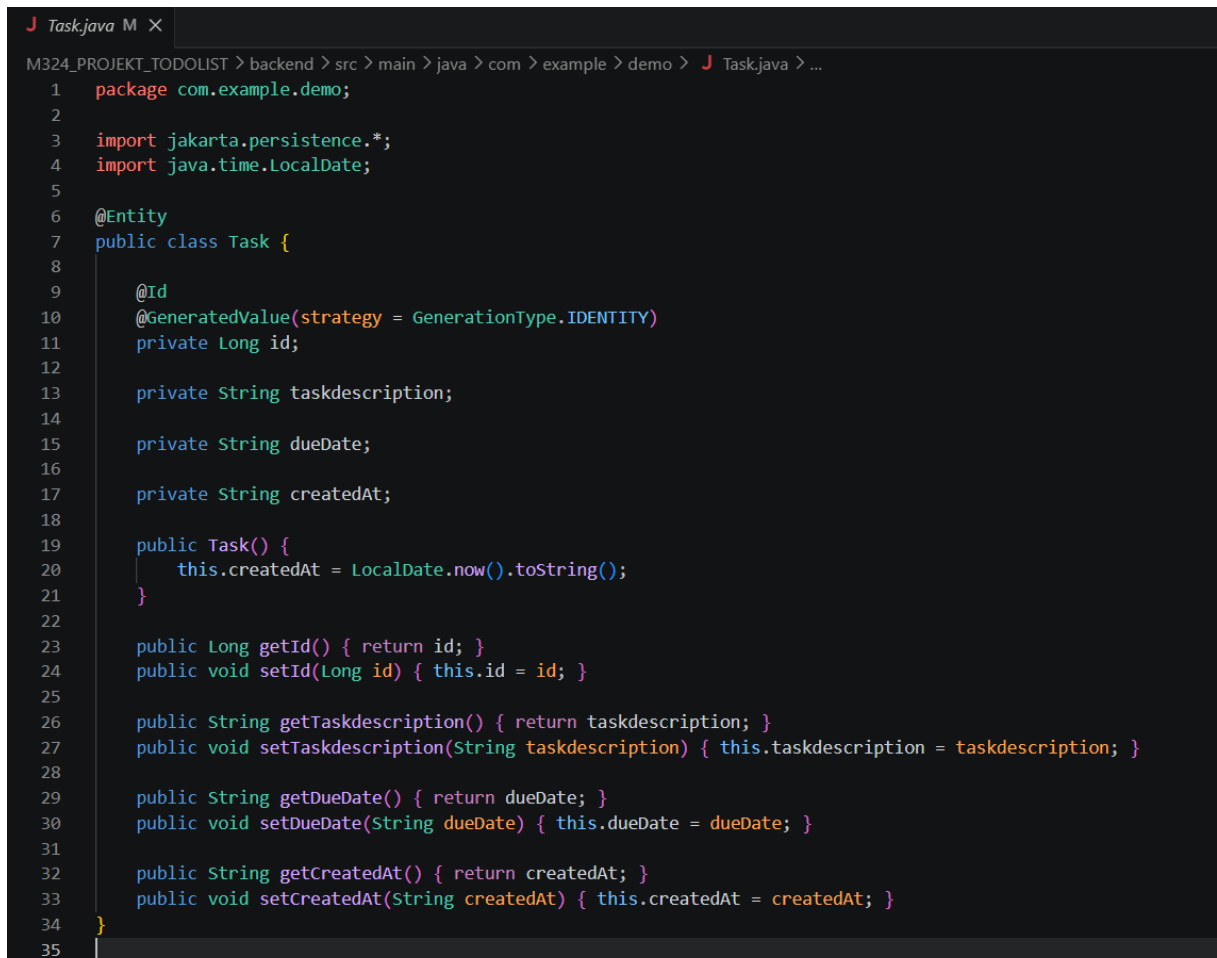
```
@Test
void testTaskHasCreationDate() {
    Task task = new Task();
    assertNotNull(task.getCreatedAt(), "Task soll ein Erfassungsdatum haben");
}
```

Dieser Test schlägt fehl, da `getCreatedAt()` in der `Task`-Klasse noch nicht existiert.

Schritt 2: Funktionalität implementieren

Die `Task`-Klasse wurde erweitert: ein `createdAt`-Feld wird beim Instanziiieren automatisch mit dem aktuellen Datum befüllt.

```
private String createdAt;  
  
public Task() {  
    this.createdAt = LocalDate.now().toString();  
}  
  
public String getCreatedAt() { return createdAt; }  
public void setCreatedAt(String createdAt) { this.createdAt = createdAt; }
```



```
J Task.java M X  
M324_PROJEKT_TODO LIST > backend > src > main > java > com > example > demo > J Task.java > ...  
1 package com.example.demo;  
2  
3 import jakarta.persistence.*;  
4 import java.time.LocalDate;  
5  
6 @Entity  
7 public class Task {  
8  
9     @Id  
10    @GeneratedValue(strategy = GenerationType.IDENTITY)  
11    private Long id;  
12  
13    private String taskdescription;  
14  
15    private String dueDate;  
16  
17    private String createdAt;  
18  
19    public Task() {  
20        this.createdAt = LocalDate.now().toString();  
21    }  
22  
23    public Long getId() { return id; }  
24    public void setId(Long id) { this.id = id; }  
25  
26    public String getTaskdescription() { return taskdescription; }  
27    public void setTaskdescription(String taskdescription) { this.taskdescription = taskdescription; }  
28  
29    public String getDueDate() { return dueDate; }  
30    public void setDueDate(String dueDate) { this.dueDate = dueDate; }  
31  
32    public String getCreatedAt() { return createdAt; }  
33    public void setCreatedAt(String createdAt) { this.createdAt = createdAt; }  
34 }  
35
```

Schritt 3:

Alle Tests bestehen

Nach der Implementierung wurden alle 4 Tests mit einem einzigen Aufruf ausgeführt:

```
mvn test
```

```

PS C:\Users\Metehan Celik\Desktop\M324-ToDo\M324_PROJEKT_TODOLIST\backend> mvn test
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.439 s -- in com.example.demo.DemoApplicationTests
[INFO] Results:
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 7.345 s
[INFO] Finished at: 2026-05-22T17:39:49+02:00
[INFO]
PS C:\Users\Metehan Celik\Desktop\M324-ToDo\M324_PROJEKT_TODOLIST\backend>

```

Applikation startet fehlerfrei

Nach allen Anpassungen wurde die Applikation gestartet, um zu prüfen ob keine Spring-Boot- oder CodingProbleme bestehen:

```
mvn spring-boot:run
```

```

PS C:\Users\Metehan Celik\Desktop\M324-ToDo\M324_PROJEKT_TODOLIST\backend> mvn spring-boot:run
[INFO] Scanning for projects...
[INFO] -----< com.example.demo >-----
[INFO] Building demo 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] >>> spring-boot:3.4.5:run (default-cli) > test-compile @ demo >>>
[INFO] --- resources:3.3.1:resources (default-resources) @ demo ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO] --- compiler:3.13.0:compile (default-compile) @ demo ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- resources:3.3.1:testResources (default-testResources) @ demo ---
[INFO] Copying 1 resource from src\test\resources to target\test-classes
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ demo ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] <<< spring-boot:3.4.5:run (default-cli) < test-compile @ demo <<<
[INFO] --- spring-boot:3.4.5:run (default-cli) @ demo ---
[INFO] Attaching agents: []

  ____ _
 / ___ \| | | |
/ /___ \| |_| |
 \___ \|____|_|
  _____|_|

:: Spring Boot ::                (v3.4.5)

```

```

PS C:\Users\Metehan Celik\Desktop\M324-Todo\M324-PROJEKT_TODO\LIST\backends > mvn spring-boot:run
2026-05-22T17:40:50.784+02:00 INFO 24308 --- [main] com.example.demo.DemoApplication : Starting DemoApplication using Java 21.0.7 with PID 24308 (C:\Users\Metehan Celik\Desktop\M324-Todo\M324-PROJEKT_TODO\LIST\backends\target\classes started by Metehan Celik in C:\Users\Metehan Celik\Desktop\M324-Todo\M324-PROJEKT_TODO\LIST\backends)
2026-05-22T17:40:50.786+02:00 INFO 24308 --- [main] com.example.demo.DemoApplication : No active profile set, falling back to 1 default profile: "default"
2026-05-22T17:40:51.253+02:00 INFO 24308 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-05-22T17:40:51.305+02:00 INFO 24308 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring data repository scanning in 44 ms. Found 1 JPA repository interface.
2026-05-22T17:40:51.639+02:00 INFO 24308 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-05-22T17:40:51.648+02:00 INFO 24308 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-05-22T17:40:51.648+02:00 INFO 24308 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.40]
2026-05-22T17:40:51.687+02:00 INFO 24308 --- [main] o.a.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-05-22T17:40:51.689+02:00 INFO 24308 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 867 ms
2026-05-22T17:40:51.880+02:00 INFO 24308 --- [main] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [name: default]
2026-05-22T17:40:51.832+02:00 INFO 24308 --- [main] org.hibernate.Version : HHH000042: Hibernate ORM core version 6.6.13.Final
2026-05-22T17:40:51.857+02:00 INFO 24308 --- [main] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-level cache disabled
2026-05-22T17:40:52.058+02:00 INFO 24308 --- [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-05-22T17:40:52.228+02:00 INFO 24308 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-05-22T17:40:52.228+02:00 INFO 24308 --- [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection conn0: url=jdbc:h2:file:./data/tododb user=SA
2026-05-22T17:40:52.244+02:00 INFO 24308 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-05-22T17:40:52.244+02:00 WARN 24308 --- [main] org.hibernate.orm.deprecation : HHH0000025: H2Dialect does not need to be specified explicitly using 'hibernate.dialect' (remove the property setting and it will be selected by default)
2026-05-22T17:40:52.262+02:00 INFO 24308 --- [main] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
Database JDBC URL [connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 2.3.232
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-05-22T17:40:52.760+02:00 INFO 24308 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-05-22T17:40:52.806+02:00 INFO 24308 --- [main] j.localContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-05-22T17:40:53.002+02:00 WARN 24308 --- [main] jpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-05-22T17:40:53.384+02:00 INFO 24308 --- [main] o.s.b.a.h2.H2ConsoleAutoConfiguration : H2 console available at '/h2-console'. Database available at 'jdbc:h2:file:./data/tododb'
2026-05-22T17:40:53.432+02:00 INFO 24308 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-05-22T17:40:53.437+02:00 INFO 24308 --- [main] com.example.demo.DemoApplication : Started DemoApplication in 2.929 seconds (process running for 3.137)

```

Was ist TDD?

Test Driven Development (TDD) ist eine Vorgehensweise beim Programmieren, bei der man zuerst den Test schreibt und erst danach den Code, der diesen Test erfüllt. Der Ablauf folgt dem ****Red-Green-Refactor**** Prinzip:

1. **Red:** Test schreiben, der fehlschlägt
2. **Green:** Minimalen Code schreiben, damit der Test besteht
3. **Refactor:** Code aufräumen, ohne dass Tests brechen

Vorteile:

- **Weniger Bugs:** Fehler werden früh entdeckt, da jede Funktion durch Tests abgesichert ist.
- **Besseres Design:** Man denkt zuerst über das Verhalten nach, bevor man Code schreibt — das führt zu klareren Schnittstellen.
- **Sicheres Refactoring:** Bestehende Tests zeigen sofort, ob eine Änderung etwas kaputt macht.
- **Lebende Dokumentation:** Die Tests beschreiben, was der Code tun soll.
- **Vertrauen in den Code:** Beim Deployen weiss man, dass die wichtigsten Funktionen funktionieren.

Nachteile:

- **Mehr Zeitaufwand am Anfang:** Tests zu schreiben kostet zusätzliche Zeit, besonders zu Beginn.
- **Lernkurve:** TDD richtig anzuwenden braucht Übung — gerade das "Test zuerst" denken ist ungewohnt.
- **Tests müssen gepflegt werden:** Bei Änderungen am Code müssen oft auch die Tests angepasst werden.
- **Nicht für alles geeignet:** Bei UI-Code oder Prototypen ist TDD oft umständlich.
- **Falsche Sicherheit:** Tests die bestehen heisst nicht, dass die Software korrekt ist — die Tests könnten unvollständig sein.